

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A

Coding Challenge

Code of Conduct:

*I Sivakumar M(192521057)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Sivakumar m
Signature of the student:

Annexure A

Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.
2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.
3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.
4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.
5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

CO4 AT 3-Industry Problem-Solving Task

1. A multinational enterprise is deploying IPSec to secure communication between its branch offices over the public internet. Analyze how Authentication Header (AH) and Encapsulating Security Payload (ESP) should be configured to ensure both integrity and confidentiality across different types of traffic. Tasks:

- A. Identify which traffic requires AH, ESP, or both**
- B. Design packet flow with appropriate IPSec modes (transport/tunnel)**
- C. Justify protocol selection for different data types**
- D. Evaluate performance vs security trade-offs**

PROGRAM:

```

import hmac
import hashlib

def generate_mac(packet, key):
    mac = hmac.new(
        key.encode(),
        packet.encode(),
        hashlib.sha256
    ).hexdigest()
    return mac

def verify_mac(packet, key, received_mac):
    calculated_mac = generate_mac(packet, key)

    if hmac.compare_digest(calculated_mac, received_mac):
        return True
    return False

# Main program
packet = input("Enter IP packet data: ")
key = input("Enter secret key: ")

# Sender generates MAC
mac = generate_mac(packet, key)

print("\n--- AH Simulation ---")
print("Original Packet:", packet)
print("Generated MAC:", mac)

# Receiver verifies MAC
received_packet = input("\nEnter received packet: ")

if verify_mac(received_packet, key, mac):
    print("Verification Successful")
    print("Packet integrity is valid.")
else:
    print("Verification Failed")
    print("Packet has been modified.")

```

OUTPUT:

```

ANSWER: C:/Users/Asus/AppData/Local/Programs/Python/Python311/000000.py
Enter IP packet data: HELLO
Enter secret key: SECRET

--- AH Simulation ---
Original Packet: HELLO
Generated MAC: 2cadb960bee4d31bed44a4ebc07f632988a147dafadc086331b074c2ccbd1b1

Enter received packet: HELLO
Verification Successful
Packet integrity is valid.

```

2. An organization faces frequent key management issues in its IPSec implementation, leading to expired or mismatched keys. Propose a solution to efficiently manage Security Associations (SA) and automate key exchange in a scalable enterprise environment.

Tasks:

A. Analyze current SA lifecycle issues

B. Design an automated key management mechanism (e.g., IKE-based)

C. Propose a strategy for key renewal and revocation

D. Evaluate scalability and fault tolerance

PROGRAM:

```
def encrypt_decrypt(data, key):
    result = ""

    for i in range(len(data)):
        result += chr(ord(data[i]) ^ ord(key[i % len(key)]))

    return result

payload = input("Enter IP packet payload: ")
key = input("Enter encryption key: ")

encrypted = encrypt_decrypt(payload, key)

print("\n--- ESP Simulation ---")
print("Original Payload:", payload)
print("Encrypted Payload:", encrypted)

decrypted = encrypt_decrypt(encrypted, key)

print("Decrypted Payload:", decrypted)
```

OUTPUT

```
RESTART: C:\OSCTIS\ASAS/Appdata/Local/110916
Enter IP packet payload: HELLO WORLD
Enter encryption key: KEY

--- ESP Simulation ---
Original Payload: HELLO WORLD
Encrypted Payload: []
Decrypted Payload: HELLO WORLD
|
```

3. A financial services company needs to implement secure email communication for sensitive transactions. Evaluate and recommend between Pretty Good Privacy (PGP) and S/MIME based on usability, scalability, and enterprise integration requirements.

Tasks:

- A. Compare PGP and S/MIME in real-world deployment scenarios**
- B. Analyze key management and trust models**
- C. Recommend a solution with justification**
- D. Design a high-level implementation workflow**

PROGRAM:

```

def display_all():
    if len(sa_table) == 0:
        print("No Security Associations available.")
        return

    print("\n--- Security Association Table ---")

    for sa in sa_table:
        sa.display()

while True:
    print("\n===== IPSec SA Management =====")
    print("1. Create SA")
    print("2. Lookup SA")
    print("3. Delete SA")
    print("4. Display All SA")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        create_sa()

    elif choice == "2":
        lookup_sa()

    elif choice == "3":
        delete_sa()

    elif choice == "4":
        display_all()

    elif choice == "5":
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")

```

OUTPUT:


```
===== IPSec SA Management =====
1. Create SA
2. Lookup SA
3. Delete SA
4. Display All SA
5. Exit
Enter choice: 1
Enter SPI: 101
Enter source IP: 192.168.1.9
Enter destination IP: 192.168.2.8
Enter algorithm: AES
Security Association created successfully.
```

```
===== IPSec SA Management =====
1. Create SA
2. Lookup SA
3. Delete SA
4. Display All SA
5. Exit
Enter choice: 2
Enter SPI to search: 101

Security Association Found
SPI: 101
Source: 192.168.1.9
Destination: 192.168.2.8
Algorithm: AES
-----
```

```
===== IPSec SA Management =====
1. Create SA
2. Lookup SA
3. Delete SA
4. Display All SA
5. Exit
Enter choice: 3
Enter SPI to delete: 101
Security Association deleted.
```

4. A company's email system is experiencing a rise in phishing and spam attacks despite basic filtering mechanisms. Analyze the weaknesses in the current system and propose an advanced spam detection strategy using email headers and content analysis.

Tasks:

- A. Identify limitations of existing filtering techniques
- B. Analyze email headers for spoofing indicators
- C. Design an improved detection model (rule-based or ML-based)
- D. Evaluate effectiveness and false positive handling

PROGRAM:

```
message = input("Enter email message: ")
key = input("Enter encryption key: ")
private_key = input("Enter private key: ")

signature = create_signature(
    message,
    private_key
)

encrypted_message = encrypt(
    message,
    key
)

print("\n--- Sender Side ---")
print("Original Message:", message)
print("Digital Signature:", signature)
print("Encrypted Message:", encrypted_message)

decrypted_message = decrypt(
    encrypted_message,
    key
)

print("\n--- Receiver Side ---")
print("Decrypted Message:", decrypted_message)

|
if verify_signature(
    decrypted_message,
    private_key,
    signature
):
    print("Digital Signature Verified.")
    print("Message is authentic and unchanged.")
```

OUTPUT:

```
===== Simplified PGP System =====
```

```
Enter email message: GOODBYE
```

```
Enter encryption key: KEY123
```

```
Enter private key: PRIVATE
```

```
--- Sender Side ---
```

```
Original Message: GOODBYE
```

```
Digital Signature: c17d8f36ee0ada2914e21dc808536983bbb1c3eb8152bd3a8d8639d61500517a
```

```
Encrypted Message: □
```

```
hupj□
```

```
--- Receiver Side ---
```

```
Decrypted Message: GOODBYE
```

```
Digital Signature Verified.
```

```
Message is authentic and unchanged.
```

5. During a security audit, it is found that encrypted email messages are vulnerable due to improper certificate handling in S/MIME. Diagnose the possible issues and recommend best practices to ensure secure certificate management and trust validation.

Tasks:

- A. Identify potential certificate-related vulnerabilities**
- B. Analyze trust chain and validation process**
- C. Propose secure certificate storage and management practices**
- D. Recommend improvements for user awareness and system configuration**

PROGRAM

```

score = 0

text = (subject + " " + body).lower()

for word in spam_words:
    if word in text:
        score += 1

links = re.findall(r"https?://", text)

if len(links) >= 2:
    score += 2

if text.count("!") >= 3:
    score += 1

if score >= 3:
    return "SPAM", score
else:
    return "LEGITIMATE", score
    .

print("==== Email Spam Classifier ====")

subject = input("Enter email subject: ")
body = input("Enter email content: ")

result, score = classify_email(
    subject,
    body
)

print("\n--- Result ---")
print("Spam Score:", score)
print("Classification:", result)

```

OUTPUT:

```

----- REGIARI. C:/Users/ASUS/AppData/LO
==== Email Spam Classifier ====
Enter email subject: YOKSO WATSUNU SOUL SOCIETY YARAA YARAA
Enter email content: HADO NUMBER 91

--- Result ---
Spam Score: 0
Classification: LEGITIMATE

```

